Write a Python program that:

1. Asks the user for a file name.
2. Reads the contents of the file.
3. Handles the following exception:
   o FileNotFoundError if the file does not exist.
4. If the file is read successfully, print its contents.
5. Include an option for the user to try again if an error occurs.

In [40]:
```
from pathlib import Path

def file_reader():
    """Takes a file path as input and prints its contents if found"""
    path = input("Enter the file path: ")
    path = Path(path)
    try:
        content = path.read_text()
    except FileNotFoundError:
        print(f"Error: File '{path}' not found.")
        user_input = input("Would you like to try again? (y/n)?").lower()
        if user_input == 'y':
            file_reader()
        else:
            return
    else:
        print(f"Contents of '{path}': ")
        print(content)

file_reader()
```

```
Error: File 'ffd' not found.
```

Write a Python program that:

1. Asks the user for a file name.
2. Prompts the user to enter some text.
3. Writes the entered text to the specified file.
4. Handles the following exceptions:
   o PermissionError if the file cannot be written to.
5. Confirm successful write by reopening the file and displaying its contents

In [27]:
```
def data_writter():
    """Asks for the file b=name and data to write into it"""
    file_name = input("Enter the file name: ")
    path = Path(file_name)
    data = input("Enter the text to save: ")

    try:
        path.write_text(data)
    except PermissionError:
        print(f"Error: Permission denied to write to '{file_name}'")
```

```
    else:
        print(f"Data saved successfully to '{file_name}'")
        print(f"Contents of '{file_name}': ")
        print(path.read_text())




data_writter()
```

```
Error: Permission denied to write to 'tryread.txt'
```

In [43]:
```
path = Path("tryread.txt")
path.write_text("Hello, World!")
```

```
---------------------------------------------------------------------------
PermissionError                           Traceback (most recent call last)
Cell In[43], line 2
      1 path = Path("tryread.txt")
----> 2 path.write_text(              )

File c:\Users\aesha\AppData\Local\Python\pythoncore-3.14-64\Lib\pathlib\__init__.py:
809, in Path.write_text(self, data, encoding, errors, newline)
    806 if not isinstance(data, str):
    807     raise TypeError('data must be str, not %s' %
    808                     data.__class__.__name__)
--> 809 with self.open(mode=   , encoding=encoding, errors=errors, newline=newline)
as f:
    810     return f.write(data)

File c:\Users\aesha\AppData\Local\Python\pythoncore-3.14-64\Lib\pathlib\__init__.py:
771, in Path.open(self, mode, buffering, encoding, errors, newline)
    769 if "b" not in mode:
    770     encoding = io.text_encoding(encoding)
--> 771 return io.open(self, mode, buffering, encoding, errors, newline)

PermissionError: [Errno 13] Permission denied: 'tryread.txt'
```

Q3

-->Note 1 text file takes string and give you string json saves the data type as it is without converting it into string

-->Note 2

Examples of error that IOError could catch:(It's like the umbrella where the file errors could)

- FileNotFoundError → raised when the file doesn't exist.
- PermissionError → raised when you don't have permission to read/write.
- IsADirectoryError → raised when you try to open a directory as if it were a file.
- NotADirectoryError → raised when a path component isn't a directory.

-->Note 3

load and dump --> works with file object

json.dump(data, path.open('w')) path.write_text(json.dumps(data))

loads and dumps --> works with strings don't need to open the file for it

json.load(path.open('r') json.loads(path.read_text())

Write a Python program that:

1. Creates a dictionary with the following structure:
2. data = { "name": "Alice", "age": 25, "city": "New York"
   }
3. Saves this dictionary into a file named data.json in JSON format.
4. Handles the following exceptions:
   o IOError if there is an error writing to the file.
5. Reloads the JSON file and displays its contents to confirm successful saving.

In [33]:
```
import json

def data_store( file_name , data):
    """Saves user data into a file in jason format"""
    path = Path(file_name + ".json")
    try:
        json.dump(data, path.open('w')) #this opens file and takes data to write in

        ## if dumps where used:
        #path.write_text( json.dumps(data) ) --> would convert data to json string

    except IOError:
        print(f"Error: {file_name} could not be written.")
    else:
        print(f"Data saved successfully to '{file_name}.json'")
        print(f"Contents of '{file_name}.json': ")
        print(json.load(path.open('r')))  # this opens file and reads data from it

        ## if load where used:
        #json.loads(path.read_text())--> would read the text from the file and then


data = {
    "name": "Alice",
    "age": 25,
    "city": "New York"
```

```
}
data_store( "data" , data )
```

```
Data saved successfully to 'data.json'
Contents of 'data.json':
{'name': 'Alice', 'age': 25, 'city': 'New York'}
```

Write a program that combines all the above steps:

1. Reads a file and prints its contents.
2. Writes user input to a new file.
3. Saves data in JSON format.
4. Validates all user inputs to ensure correctness.
5. Ensures all exceptions are handled gracefully.
6. Provides a menu system to navigate between the tasks.

In [36]:

```
def combination_function():
    """  Reads a file
        ,write user input to a file
        ,saves user data in a json format
        ,Validates all user inputs to ensure correctness.
        ,Ensures all exceptions are handled gracefully.
        ,Provides a menu system to navigate between the tasks.
    """
    user_choice = input("Choose an option:\n" \
                        "1. Read a file\n" \
                        "2. Write to a file\n" \
                        "3. Store data to JSON \n" \
                        "4. Exit\n" \
                        "Choose an option (1, 2, 3 or 4): ")
    if user_choice == '1':
        file_reader()
    elif user_choice == '2':
        data_writter()
    elif user_choice == '3':
        data_store_bonus()
    elif user_choice == '4':
        print("Goodbye!")
        return
    combination_function()


combination_function()
```

```
Contents of 'read.txt':
My name is Aiysha
Error: File 'data.txt' not found.
Error: File 'f' not found.
Contents of 'read.txt':
My name is Aiysha
Data saved successfully to 'read.txt'
Contents of 'read.txt':
Hello, world!
Goodbye!
```

Write a Python program that:

1. Prompts the user to enter their name, age, and email.
2. Validates the inputs as follows:

   o Name must only contain alphabetic characters and spaces.

   o Age must be a positive integer.

   o Email must follow a standard email format (e.g., contain "@" and ".").
3. Handles the following exceptions:
   o ValueError if the age is not an integer.
4. Outputs the validated data.

In [122…

```
################ to check email validity ##################
def email_check(email):
    """Validate the email format entered by the user"""
    while(
        "@" not in email or
        "." not in email.split("@")[-1] or
        email.count("@") != 1 or
        not any(ch.isalpha() for ch in email.split("@")[-1]) or ## this two rows to
        not any(ch.isalnum() for ch in email.split("@")[0]) or     ## and after @ "
        email.split("@")[0].startswith(".") or
        email.split("@")[-1].endswith(".") or
        ".." in email or
        "@." in email or
        ".@" in email

    ):
        print("ERROR: Enter a valid email address.")
        email = input("Enter your email or 'q' to quit: ")
        if email.lower() == 'q':
            return None

################ to check name validity ####################
def user_name_check(name):
    """Validate the user name format entered by the user"""
    while not name.isalpha():
        print("ERROR: Enter alphabetic characters only.")
        name = input("Enter your name or 'q' to quit: ")
        if name.lower() == 'q':
            return None
    return name

################ to check age validity ####################
def age_check(age):
    """Validate the user age format entered by the user"""
    try:
        age = int(age)
```

```
    except ValueError:
        print("ERROR:Age must be an integer(ValueError)!")
        age = input("Enter your age: ")
    else:
        while  not str(age).isdigit() or age <= 0:
            print("ERROR:Age must be a valid positive integer!")
            age = input("Enter your age or 'q' to quit: ")
            if age.lower() == 'q':
                return None
    return age


######### To validate all user inputs #########
def validate_user_input():
    """Validates user inputs for name, age, and email"""
    # Validate name (alphabetic characters only)
    name = input("Enter your name or 'q' to quit: ")
    if name.lower() != 'q':
       name = user_name_check(name)
    else:
        return


    # Validate age (positive integer)
    age = input("Enter your age or 'q' to quit: ")
    if age.lower() != 'q':
        age = age_check(age)
    else:
        return

    # Validate email by calling email_check function
    email = input("Enter your email or 'q' to quit: ")
    if email.lower() != 'q':
        email = email_check(email)
    else:
        return

    print(f"Name: {name}\nAge: {age}\nEmail: {email}")


validate_user_input()
```

Draft

In [ ]:

```
email1= "fnf.fkodnf@gmail.com"
email2= ".@"
email= "d.f@.com"

if (
    "@" not in email or
    "." not in email.split("@")[-1] or
      email.count("@") != 1 or
      not any(ch.isalpha() for ch in email.split("@")[-1]) or ## this two rows to c
      not any(ch.isalnum() for ch in email.split("@")[0]) or
      email.split("@")[0].startswith(".") or
```

```
      email.split("@")[-1].endswith(".") or
      ".." in email or
      "@." in email or
      ".@" in email

):

    print("ERROR: Enter a valid email address.")
```

Bonus Task

Enhance Problem 3 by allowing the user to input their own data (e.g., name, age, and city) instead of using hardcoded values. Include validation to ensure:

1. Name is a string.
2. Age is a positive integer.
3. City is a non-empty string.

In [125…

```
import json

def data_store_bonus( file_name = "data.json" , data={}):
    """Saves user data into a file in jason format"""
    path = Path(file_name + ".json")
    if data == {}:
        name = input("Enter name or 'q' to quit: ")
        if name.lower() == 'q':
            return None
        name = user_name_check(name)
        data['name'] = name

        age = input("Enter age or 'q' to quit: ")
        if age.lower() == 'q':
            return None
        age = age_check(age)
        data['age'] = age

        city = input("Enter city: ")
        while city == "":
            print("ERROR: City cannot be empty.")
            city = input("Enter city or 'q' to quit: ")
            if city.lower() == 'q':
                return None

        data['city'] = city

    try:
        json.dump(data, path.open('w')) #this opens file and takes data to write in
    except IOError:
        print(f"Error: {file_name} could not be written.")
    else:
        print(f"Data saved successfully to '{file_name}'")
        print(f"Contents of '{file_name}': ")
        print(json.load(path.open('r')))  # this opens file and reads data from it
```

```
data_store_bonus()
```

```
ERROR:Age must be an integer(ValueError)!
Data saved successfully to 'data.json'
Contents of 'data.json':
{'name': 'dd', 'age': '5', 'city': 'alex'}
```